Pencil Cup: Architecture, Implementation and Engineering Analysis

ABProjects · Technical Report · 17 October 2026

Abstract. A hand-drawn 3v3 arcade football game featuring human and AI teams, a fixed-step match simulation, possession, goalkeeping, replay, set pieces, and a customizable presentation. This report examines the actual shipped single-file architecture, execution model, engineering trade-offs, targeted improvements and validation limits. Statements about changes are grounded in the code supplied with this release. Performance improvements are presented as structural reductions in eligible work, not unmeasured frame-rate claims.

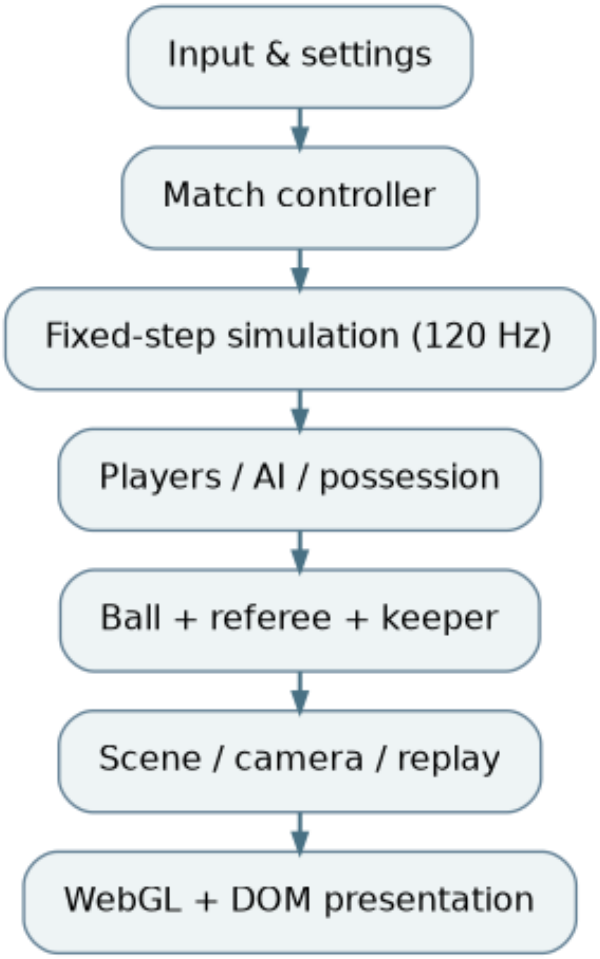


Figure 1: System architecture

Figure 1. Functional system view; all runtime subsystems remain within one self-contained HTML document.

Keywords: HTML5, WebGL, Three.js, browser games, frame scheduling, game engineering, deterministic execution, source preservation.

1. Problem and gameplay model

Pencil Cup implements a complete short-form football match within a single browser document. The match combines player-directed movement, autonomous teammates, opposing AI, ball possession, passing, kicking, goalkeeping, set pieces and results. Three game modes are exposed in the authored menu: friendly play, a quick match, and the Pencil Cup competition. Interaction is intentionally direct rather than simulation-heavy; responsiveness and legibility of movement take priority over full professional-football rules.

2. Runtime architecture

The HTML contains the CSS, UI, bundled Three.js renderer, simulation classes, sound logic and startup sequence. Rendering uses a Three.js scene and camera. The application controller coordinates the match, input, scene, audio, camera, replay and HUD. Input is collected continuously; simulation runs at a fixed rate; scene transforms and overlays are updated from the current match state. No CDN, imported runtime module or separately fetched art asset is required by the uploaded standalone HTML build. The shipped, minified bundle is a distribution artifact, not a claim that all source identifiers are human-readable.

3. Fixed-step dynamics

The source declares `fixedStep = 1/120` second with at most 14 fixed simulation substeps per animation frame. It accumulates frame delta and advances in fixed quanta, bounding the backlog to prevent runaway catch-up after stalls. A real-time frame step `dt` therefore contributes approximately `floor(accumulator/dt_fixed)` physics updates, limited by the step cap. The published constants also include gravity 21.5 world units/s², ball radius 0.31 world units and a ground-bounce factor of 0.56. These are gameplay parameters, not measurements of real football. Ball and player positions update from fixed-step state, while visualization runs at the browser display cadence.

4. Player control, artificial opponents and match rules

The controller processes keyboard, mouse and optionally gamepad input. Play-level state tracks the controlled player, the ball, attacking direction, kick state, possession and referee/game phases. The AI and character controllers operate on the same simulated objects; there is no server authoritative state. Goalkeeper actions, kickoff ownership, the currently allowed set piece, and pause/confirmation transitions are treated as explicit match states. The replay recorder stores visual samples during play and can temporarily replace ordinary scene pose updates for replay presentation. This mechanism explains why replay completion and pause must be treated as distinct transitions.

5. Rendering and user interface

The system uses one WebGL canvas alongside DOM-based cards for menu, pause, settings and results. The renderer supports three user-facing quality tiers and caps pixel ratio according to display DPR and a tier-specific ceiling: 1 for low, approximately 1.8 for default, and 2 for high. The application also includes reduced-motion and legibility settings. The new idle-render gate is placed after input/state advancement: active simulation still calls the render path every animation frame; non-simulation views render at no more than 10 Hz, and hidden tabs skip WebGL draws. This saves GPU submissions when the football simulation is not progressing without changing its fixed-step numerical update.

6. Correctness boundaries and evaluation

The application already pauses active gameplay on blur or visibility loss. The revision preserves this rule and does not change the numerical parameters, player movement, AI decisions, ball behavior, match rules or audio synthesis. JavaScript syntax and single-document packaging were verified automatically. The game initialized in virtual-desktop Chromium, successfully advanced from team selection into match setup, and reached the kickoff sequence without an uncaught JavaScript error. Full-match and device regression coverage remains outstanding; no FPS uplift or absence-of-regressions claim is made. A browser acceptance matrix is shipped with the repository.

7. Simulation invariants and cost model

The main browser scheduler (`frame`) separates the variable-rate display callback from the discrete physics step. If `h = 1/120` s and a frame arrives with real-time delta `d`, the accumulator evolves as `a <- min(a + d, 14h)`. For each `a >= h`, the engine advances the match state once and subtracts `h`, stopping after fourteen steps. The visible frame interval is clamped to at most 0.1 s. This prevents a resumed tab from demanding an arbitrarily large simulation catch-up burst, while preserving deterministic step size for ordinary continuous play. Exact match replay determinism still depends on identical input/state timing and pseudorandom initialization; fixed-step integration alone does not guarantee a bit-for-bit replay.

The render-resolution setting has a quadratic relationship with fragment workload. If a display has logical width `W` and height `H`, a renderer pixel ratio `r` produces approximately $W * H * r^2$ shaded pixels before post-processing and geometry effects. Choosing `r = 2` instead of `r = 1` therefore has a potential fourfold pixel-count cost. The existing low/default/high bounds let players trade sharpness for graphics throughput without changing simulation precision. The release does not retune these bounds.

The new render governor is deliberately excluded from the deterministic simulation path. On a 60 Hz monitor, an inactive scene could previously submit roughly 60 draws per second; an elapsed-time threshold of 100 ms permits at most around 10. The number of animation-frame callbacks is unchanged, and the game still calls `advance` on every callback. No optimization is claimed for active-play FPS, and the reduction in idle GPU utilization is an expectation to verify on real hardware, not a measurement.

8. Failure modes and engineering decisions

Focus and visibility. The game already routes browser blur and hidden-tab events to `pauseForFocusLoss`, avoiding continued active play without user input. The revised renderer additionally skips hidden draws. When the window regains focus, ordinary animation frames continue and the user can resume through the existing UI.

Input consistency. Shortcuts and pointer actions are processed through the centralized input controller. The governor was placed *after* controller advance specifically so a pause/resume command is not delayed by reduced menu rendering. This is safer than lowering `requestAnimationFrame` frequency globally.

High-latency frames. The accumulator cap and maximum delta limit the work after long stalls. Increasing those limits to "make the simulation catch up" would increase spikes and alter gameplay; the release keeps them unchanged.

Distribution. The bundled single-document code is minified and includes a third-party engine. Reformatting or transpiling the entire bundle would create a large, difficult-to-review diff with high regression risk. Readability is improved through documentation, named entry points and explicit comments around the one new runtime behavior, rather than replacing the bundled runtime with a new architecture.

9. Verification plan for a production release

A controlled test should compare old and new builds using identical team selection, match duration, opponent mode, display resolution and browser version. At minimum: a five-minute active match including two possession switches, a charged shot, a goal and a restart; pause/restart/quit navigation; keyboard/gamepad focus switching; a full cup tournament flow; and deliberate backgrounding for thirty seconds. A source-backed acceptance suite should record match clock before/after pause, frame-time distributions, memory usage and unexpected console errors. A visual comparison should include player/ball transform, kick trajectory, score, control hints and overlay state. These checks matter more than an unsupported "optimized by X%" claim.

Controlled browser observations

In a visible Chromium session under Xvfb/SwiftShader, both baseline and patched files initialized a 1280×720 WebGL canvas and loaded the team-selection menu without uncaught JavaScript errors. The patched version advanced to match

setup with six players and a 0–0 score and displayed the kick-off overlay. For an isolated scheduling check, the renderer's draw function was replaced with a counter while the original animation frame scheduler ran: the baseline made **135** idle render requests versus **26** for the patch during a roughly 3.6-second observation. These are *draw requests* rather than completed GPU renders, and the test does not measure real graphics performance.

10. Build structure and release integrity

The playable deliverable remains `index.html`. No source was split into imports, ES modules, bundles or fetched runtime resources. Documentation, diagnostics and GitHub publishing helpers are external to the playable game; they are not runtime dependencies. Original and updated SHA-256 values are recorded in `docs/RELEASE_MANIFEST.json`. The applied behavior changes are intentionally isolated and documented in `docs/RELEASE_NOTES.md`.

The included `tests/verify.mjs` checks that the repository contains a self-contained HTML document, runs JavaScript syntax validation, and confirms that the expected safety/performance markers are present. It is a smoke check, not a substitute for end-to-end playtesting on real graphics hardware.

11. Verification status

Completed: source inspection; successful parsing of every inline script using Node.js; HTML structural check; traceable, limited source patches; independent archive integrity checks. **Not completed**: GPU-accelerated frame benchmarks, exhaustive input-device testing, multiplayer/network testing (not applicable), long-run performance sampling, and full in-game visual regression. Initial headless Chromium could not initialize WebGL, but a virtual-desktop Chromium instance using SwiftShader subsequently loaded the game. Startup/menu smoke checks succeeded without uncaught JavaScript exceptions. No exhaustive game-playthrough, cross-browser matrix or GPU performance benchmark was performed.

12. Deployment and third-party notices

The playable build is distributed as one `index.html`. Local testing can be performed by opening it in a desktop browser; if browser security policies restrict a feature, serve the same file from a local HTTP server. Publishing to itch.io as an HTML5 game requires placing `index.html` at the root of the uploaded ZIP. See the game's itch.io listing for the existing public game page and any project imagery. Repository screenshots must be captured or downloaded and checked for permission before embedding; this release does not invent gameplay imagery.

The file contains embedded third-party engine/runtime code and licenses/attribution already carried inside its source. A public GitHub repository does **not** grant additional reuse rights by itself. No new overarching licence has been selected on the author's behalf.

References

1. ABProjects, Pencil Cup on itch.io, live project listing.
2. ABProjects, `index.html`, source release analyzed for this report.
3. Three.js, WebGL rendering library, embedded release code and notices.
4. ABProjects, `docs/RELEASE_MANIFEST.json` and `tests/verify.mjs`, release verification evidence.